GUÍA DE USO / 01

Del LOGO al ESP32

Guía para alumnos · CHQLogoWeb 1.5.0 · CHQ Labs

En estas prácticas vas a leer valores del LOGO y enviarle valores desde un ESP32. La librería se comunica con el PLC por su portal web; tú trabajas con instrucciones sencillas como readM, readAM y setM.

Primero: ¿qué es una marca?

Marca	Qué guarda	Ejemplo
M: digital	Un estado: 0 o 1, apagado o encendido.	M1 puede indicar si hay una condición activa.
AM: analógica	Un número entero.	AM2 puede guardar una distancia de 150 cm.

M1 y AM1 son marcas distintas. Las marcas están dentro del LOGO. Una marca no es una salida física: para accionar Q1, tu programa del PLC debe relacionar una marca, por ejemplo M2, con esa salida.

Prepara tu práctica

Necesitas el ESP32, el LOGO con su portal web habilitado y una red donde ambos puedan comunicarse. Abre primero el portal del LOGO desde la PC y comprueba sus valores.

PlatformIO: copia la carpeta CHQLogoWeb dentro de lib/ de tu proyecto.
Arduino IDE: instala el ZIP desde Programa > Incluir librería > Añadir biblioteca .ZIP. Selecciona tu placa y su puerto.

Dato que debes editar	Qué significa
WIFI_SSID	Nombre de tu red Wi-Fi.
WIFI_PASSWORD	Contraseña de esa red Wi-Fi.
LOGO_IP	Dirección del PLC, por ejemplo 192.168.1.3.
LOGO_PASSWORD	Contraseña del portal web del LOGO.

Cómo seguir la guía

Empieza por el programa completo de la página 2 y su explicación en la 3. Después elige: tres digitales (p. 4), una analógica (p. 5), tres analógicas (p. 6) o combinarlas (p. 7). Las escrituras y el sensor simulado están en las páginas 8 y 9.

Abre el monitor serie a 115200 baudios. Los ejemplos usan &Serial para mostrar la conexión y los valores cuando cambian.

GUÍA DE USO / 02

Tu primer programa

Abre el ejemplo Simple. Edita las cuatro constantes de conexión y carga el programa en el ESP32. Este ejemplo lee M1 y M2.

```
#include <WiFi.h>
#include <CHQLogoWeb.h>

const char* WIFI_SSID = "TU_WIFI";
const char* WIFI_PASSWORD = "TU_CLAVE_WIFI";
const char* LOGO_IP = "192.168.1.3";
const char* LOGO_PASSWORD = "TU_CLAVE_WEB";

CHQLogoConfig config(LOGO_PASSWORD, "Web User", &Serial);
CHQLogoWeb logo(LOGO_IP);
bool m1 = false, m2 = false;

void setup() {
  Serial.begin(115200);
  WiFi.setAutoReconnect(true);
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
  while (WiFi.status() != WL_CONNECTED) delay(250);
  WiFi.setSleep(false);
  while (!logo.begin(config)) { Serial.println(logo.lastError()); delay(1000); }
  while (!logo.waitUntilReady()) { Serial.println(logo.lastError()); delay(1000); }
  Serial.println("LOGO listo. Envia 1 para activar M2 o 0 para apagarla.");
}

void loop() {
  logo.loop();
  m1 = logo.readM(1);
  m2 = logo.readM(2);

  if (Serial.available()) {
    const char tecla = static_cast<char>(Serial.read());
    if (tecla == '1' || tecla == '0') logo.setM(2, tecla == '1');
  }
}
```

¿Qué debes ver?

Primero espera a que aparezca LOGO listo. Después verás los estados de M1 y M2. Si un valor permanece igual, la librería no vuelve a imprimirlo; lo muestra cuando cambia.

Envía 1 desde el monitor para pedir M2=1 y 0 para pedir M2=0. M1 solo se lee. Si M2 controla Q1 en tu programa del LOGO, también podrás observar el cambio de esa salida.

Las variables m1 y m2 guardan los estados para usarlos en tu programa del ESP32. Su nombre puede cambiar; el número dentro de readM es el que elige la marca del PLC.

GUÍA DE USO / 03

Qué hace cada parte

setup se ejecuta una vez al arrancar. loop se repite mientras el ESP32 está encendido.

1. Preparar la comunicación

Instrucción	Explicación
#include <WiFi.h>	Permite conectar el ESP32 a la red Wi-Fi.
#include <CHQLogoWeb.h>	Permite usar las instrucciones de esta librería.
CHQLogoConfig config(...)	Guarda la clave web y las opciones del LOGO. &Serial activa los mensajes.
CHQLogoWeb logo(LOGO_IP);	Crea el objeto logo, que usarás para hablar con ese PLC.
Serial.begin(115200);	Inicia los mensajes del monitor serie a esa velocidad.
WiFi.begin(...)	Empieza la conexión Wi-Fi. La línea while espera a que se conecte.
WiFi.setAutoReconnect(true);	Activa la reconexión Wi-Fi del ESP32.
WiFi.setSleep(false);	Desactiva el ahorro Wi-Fi para esta práctica.

2. Iniciar el LOGO: dos llamadas

Instrucción	Explicación
logo.begin(config)	Prepara la librería con los datos de conexión.
logo.waitUntilReady()	Espera a entrar al LOGO y recibir una lectura nueva de M1.

Los while del ejemplo repiten el intento si hace falta. Si no hay red o la contraseña es incorrecta, setup permanece esperando. El Wi-Fi se conecta con WiFi.h; begin no lo conecta por ti.

3. Leer dentro de loop

```
logo.loop();       // Atiende la conexion y actualiza las marcas.
m1 = logo.readM(1); // Guarda el ultimo estado de M1 en m1.
```

logo.loop debe ejecutarse continuamente. readM entrega el último valor recibido; no espera una consulta nueva cada vez que lo llamas. Usa availableM(1) para saber si ese dato es válido. Para AM existe availableAM.

bool guarda true/false, equivalentes a 1/0. int16_t guarda un número entero. Una variable de tu programa no crea una marca ni modifica el programa del LOGO.

Para las teclas, Serial.available() revisa si llegó algo y Serial.read() lee el carácter recibido. El if acepta solo 1 o 0 y elige qué valor pedir con setM.

GUÍA DE USO / 04

Leer tres marcas digitales

Usa el programa de la página 2 como base. Aquí solo vas a leer M1, M2 y M3; el Wi-Fi y el inicio del LOGO se conservan.

Paso 1. Declara las tres variables

Reemplaza la declaración de m1 y m2 por esta línea. Va antes de setup:

```
bool m1 = false, m2 = false, m3 = false;
```

Paso 2. Reemplaza todo el loop

```
void loop() {
  logo.loop();
  m1 = logo.readM(1);
  m2 = logo.readM(2);
  m3 = logo.readM(3);
}
```

Esta línea...	Guarda en el ESP32...
m1 = logo.readM(1);	El estado de la marca M1.
m2 = logo.readM(2);	El estado de la marca M2.
m3 = logo.readM(3);	El estado de la marca M3.

Paso 3. Comprueba los cambios

Haz que M1, M2 y M3 cambien desde el programa del LOGO o desde su portal. Observa los mensajes del monitor serie. En este loop no hay setM, así que el ESP32 no escribe esas marcas.

El número elige la marca: readM(3) lee M3. No significa “leer tres marcas”. Para elegir M5 cambia esa llamada por readM(5). Puedes llamar a tu variable m3, boton o permiso; el número sigue siendo el que manda.

¿Ya llegaron las tres lecturas?

```
// Si necesitas usar las tres juntas, agrega esto tras las lecturas:
if (logo.availableM(1) && logo.availableM(2) && logo.availableM(3)) {
  // Aqui usa m1, m2 y m3 en tu programa.
}
```

La espera de setup solo comprobó M1. Las otras marcas reciben su primera lectura al avanzar loop. && significa que se deben cumplir las tres condiciones.

GUÍA DE USO / 05

Leer un valor analógico

Abre LecturaAnalogica. Este programa guarda en am2 el número que tenga AM2 en el LOGO. Solo hace lecturas.

```
#include <WiFi.h>
#include <CHQLogoWeb.h>

const char* WIFI_SSID = "TU_WIFI";
const char* WIFI_PASSWORD = "TU_CLAVE_WIFI";
const char* LOGO_IP = "192.168.1.3";
const char* LOGO_PASSWORD = "TU_CLAVE_WEB";

CHQLogoConfig config(LOGO_PASSWORD, "Web User", &Serial);
CHQLogoWeb logo(LOGO_IP);
int16_t am2 = 0;

void setup() {
  Serial.begin(115200);
  WiFi.setAutoReconnect(true);
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
  while (WiFi.status() != WL_CONNECTED) delay(250);
  WiFi.setSleep(false);
  config.profile.readAnalog = true;
  while (!logo.begin(config)) { Serial.println(logo.lastError()); delay(1000); }
  while (!logo.waitUntilReadyAM(0, 2)) { Serial.println(logo.lastError()); delay(1000); }
  Serial.println("LOGO listo. Lectura automatica de AM2.");
}

void loop() {
  logo.loop();
  am2 = logo.readAM(2);
}
```

Las tres diferencias respecto a una digital

Cambio	Para qué sirve
int16_t am2 = 0;	Crea una variable para guardar el número recibido.
config.profile.readAnalog = true;	Habilita la lectura de marcas AM antes de begin.
logo.waitUntilReadyAM(0, 2)	Espera una lectura nueva de AM2. El 0 indica que espera sin límite de tiempo.

am2 = logo.readAM(2); guarda el último número recibido de AM2. Prueba 30, 100 y 200 en el portal y comprueba que el ESP32 los sigue.

AM guarda enteros de -32768 a 32767. El significado lo define tu práctica: 150 puede representar 150 cm si así lo usas. La librería no convierte automáticamente unidades ni decimales.

GUÍA DE USO / 06

Leer tres marcas analógicas

Usa el programa de la página 5 como base. Ahora leerás AM1, AM2 y AM3 y les darás nombres relacionados con la práctica.

Paso 1. Declara las variables antes de setup

Reemplaza int16_t am2 = 0 por:

```
int16_t nivel = 0, temperatura = 0, distancia = 0;
```

Paso 2. Mantén las AM habilitadas

Esta línea debe permanecer dentro de setup, antes de logo.begin(config):

```
config.profile.readAnalog = true;
```

En la segunda llamada de inicio usa AM1 como primera marca a comprobar:

```
while (!logo.waitUntilReadyAM(0, 1)) {
  Serial.println(logo.lastError());
  delay(1000);
}
```

Paso 3. Reemplaza todo el loop

```
void loop() {
  logo.loop();
  nivel = logo.readAM(1);
  temperatura = logo.readAM(2);
  distancia = logo.readAM(3);
}
```

Marca del LOGO	Variable del ESP32	Posible uso en tu práctica
AM1	nivel	Un nivel medido en centímetros.
AM2	temperatura	Una temperatura expresada como entero.
AM3	distancia	Una distancia medida en centímetros.

Los nombres no hacen una conversión: si AM1 vale 120, nivel guarda 120. Tú decides qué representa ese número y debes usar el mismo criterio en el programa del LOGO.

Para otra marca, cambia el número: readAM(5) lee AM5. waitUntilReadyAM comprueba solo la marca elegida; las demás se actualizan después. Usa availableAM(1), availableAM(2) y availableAM(3) si necesitas comprobar que ya llegaron las tres.

GUÍA DE USO / 07

Combinar digitales y analógicas

Puedes leer M y AM en el mismo programa. Este ejemplo lee tres marcas en total: M1, M2 y AM2.

Paso 1. Usa Simple como base y cambia las variables

```
bool motor = false, permiso = false;
int16_t distancia = 0;
```

Colócalas antes de setup, en lugar de la declaración original de m1 y m2.

Paso 2. Habilita las AM antes de begin

```
config.profile.readAnalog = true;
```

Conserva el resto de setup del ejemplo Simple, incluyendo waitUntilReady(). Esa llamada comprobará M1 al arrancar.

Paso 3. Reemplaza todo el loop

```
void loop() {
  logo.loop();
  motor = logo.readM(1);
  permiso = logo.readM(2);
  distancia = logo.readAM(2);
}
```

Variable	Se obtiene de...	Tipo
motor	M1	bool: encendido o apagado.
permiso	M2	bool: verdadero o falso.
distancia	AM2	int16_t: número entero.

Cómo agregar una cuarta marca

Declara otra variable del tipo adecuado y añade una lectura. Por ejemplo, para leer también AM3:

```
// Antes de setup:
int16_t nivel = 0;

// Dentro del loop, despues de logo.loop():
nivel = logo.readAM(3);
```

La librería admite 16 marcas diferentes en total, sumando M y AM y contando también las que escribes. Leer varias veces la misma marca no ocupa otro lugar. El perfil inicial permite elegir M1..M32 y AM1..AM8.

Las marcas se consultan por turno. Agregar marcas aumenta el trabajo de comunicación; los valores no llegan todos exactamente al mismo instante.

GUÍA DE USO / 08

Enviar un valor al LOGO

Leer trae un valor al ESP32. Escribir pide al LOGO que guarde un valor. Esa petición se atiende mientras tu programa sigue llamando a logo.loop().

Encender o apagar una marca digital

```
logo.setM(2, true);   // Solicita M2 = 1.
logo.setM(2, false);  // Solicita M2 = 0.
```

Son dos alternativas: usa una según lo que quieras hacer. No las pongas seguidas para intentar producir un pulso. Si pides dos valores antes de que la librería envíe, puede conservar solo el último.

En Simple, la tecla 1 elige true y la tecla 0 elige false. Q1 cambiará únicamente si el programa del LOGO la controla a partir de M2.

Enviar un número a una marca analógica

Antes de begin habilita lectura y escritura AM:

```
config.profile.readAnalog = true;
config.profile.writeAnalog = true;
```

Dentro de loop puedes pedir un valor con una sola línea:

```
logo.setAM(2, 150);  // Solicita AM2 = 150.
```

Si tu práctica representa centímetros, ese 150 equivale a 150 cm. Para enviar la medida de una variable usarías logo.setAM(2, distancia). La marca debe estar reservada para este escritor durante la prueba.

Cómo saber si se aplicó

La librería intenta el envío y luego vuelve a leer. Matched indica que esa lectura coincide con lo solicitado. Un true devuelto por setM o setAM solo indica que la petición fue aceptada en el ESP32.

Repetir el mismo set no manda el valor una y otra vez. Si hay un error, la librería tampoco repite automáticamente una escritura ya intentada. Revisa el valor y los mensajes antes de decidir qué hacer.

Qué hace el sensor simulado de la siguiente página

Empieza en 30 cm, sube de 10 en 10 hasta 200 y luego baja. Cada valor se pide con setAM(2, distanciaCm). Solo avanza después de comprobar el anterior y de que transcurra el intervalo de 1000 ms.

No hay un sensor físico conectado. La práctica sirve para observar cómo llega un número al PLC y cómo puede usarlo tu programa del LOGO.

GUÍA DE USO / 09

Práctica: distancia simulada

Abre UltrasonicoSimulado. Edita los datos de conexión y reserva AM2 para el ESP32. Observa la distancia en el portal del LOGO.

```
#include <WiFi.h>
#include <CHQLogoWeb.h>

const char* WIFI_SSID = "TU_WIFI";
const char* WIFI_PASSWORD = "TU_CLAVE_WIFI";
const char* LOGO_IP = "192.168.1.3";
const char* LOGO_PASSWORD = "TU_CLAVE_WEB";

CHQLogoConfig config(LOGO_PASSWORD, "Web User", &Serial);
CHQLogoWeb logo(LOGO_IP);
int16_t distanciaCm = 30;
int16_t pasoCm = 10;
uint32_t ultimoCambio = 0;
const uint32_t INTERVALO_MS = 1000;

// La unidad probada aplica la escritura AM aunque responde e=16.
// Usa 0 si tu equipo no presenta ese comportamiento.
const uint16_t CODIGO_AM_POR_LECTURA = 16;

void setup() {
  Serial.begin(115200);
  WiFi.setAutoReconnect(true);
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
  while (WiFi.status() != WL_CONNECTED) delay(250);
  WiFi.setSleep(false);
  config.profile.readAnalog = true;
  config.profile.writeAnalog = true;
  config.profile.analogWriteReadbackCode = CODIGO_AM_POR_LECTURA;
  while (!logo.begin(config)) { Serial.println(logo.lastError()); delay(1000); }
  while (!logo.waitUntilReadyAM(0, 2)) { Serial.println(logo.lastError()); delay(1000); }
  Serial.println("Sensor simulado: 30 a 200 cm en AM2. Reserva esa marca para el ESP32.");
}

void loop() {
  logo.loop();
  if (!logo.availableAM(2)) return;

  if (logo.writeStateAM(2) == CHQLogoWriteState::Matched &&
      static_cast<uint32_t>(millis() - ultimoCambio) >= INTERVALO_MS) {
    ultimoCambio = millis();
    distanciaCm += pasoCm;
    if (distanciaCm == 200 || distanciaCm == 30) pasoCm = -pasoCm;
  }
  logo.setAM(2, distanciaCm);
}
```

distanciaCm guarda el valor; pasoCm indica cuánto sube o baja. INTERVALO_MS es el tiempo deseado entre cambios. millis mide el tiempo sin detener todo el programa; return termina esa vuelta de loop si aún no hay un dato válido.

La unidad probada responde con código 16 aunque cambia AM2. El ejemplo lo tiene configurado para comprobar por lectura. En otro equipo conserva 0 hasta que el profesor confirme ese comportamiento. Si no se comprueba un valor, el ejemplo no avanza al siguiente.

GUÍA DE USO / 10

Comprueba lo que aprendiste

Haz las prácticas en orden. Antes de cambiar de ejercicio, comprueba en el portal y en el monitor serie que estás trabajando con la misma marca.

Tres ejercicios para clase

Ejercicio	Qué debes cambiar	Qué debes comprobar
1. Tres digitales	Lee M1, M2 y M3 con el ejemplo de la página 4.	Al cambiar una, identificas qué variable del ESP32 la sigue.
2. Tres analógicas	Lee AM1, AM2 y AM3. Prueba valores distintos: 30, 100 y 200.	Cada variable muestra el número de su propia marca.
3. Lectura mixta	Lee M1, M2 y AM2. Después agrega AM3.	Usas bool para estados e int16_t para números.

Dudas frecuentes

Si pasa esto...	Revisa esto
No sale de setup	Comprueba el Wi-Fi, la IP del LOGO y la contraseña web. El ejemplo espera hasta tener conexión y una primera lectura.
Solo imprime una vez	Es normal si el valor no cambia. Con &Serial se muestran la primera lectura y los cambios.
Veo cero	Puede ser un cero real o todavía no haber lectura válida. Comprueba availableM o availableAM.
readAM no actualiza	Habilita config.profile.readAnalog antes de begin y comprueba el número de marca.
Agregué otro setup o loop	Debe haber uno de cada uno. Al adaptar una práctica, reemplaza el bloque indicado.
Puse un delay largo	Durante ese tiempo no llamas a logo.loop y la comunicación deja de atenderse. Usa millis, como en el sensor simulado.

Cinco instrucciones que vas a usar mucho

```
logo.loop();                // Atiende la comunicacion.
m1 = logo.readM(1);          // Guarda el estado de M1.
distancia = logo.readAM(2);  // Guarda el numero de AM2.
logo.setM(2, true);          // Solicita encender M2.
logo.setAM(2, 150);          // Solicita AM2 = 150.
```

Estos ejemplos parten de las pruebas con un LOGO! 8 y un ESP32-S3. Si el equipo de tu práctica responde de otra forma, consulta al profesor para ajustar la configuración.